DEBI INDUSTRY GRADUATION PROJECT | TECHNICAL DOCUMENTATION

Adaptive Learning Assistant

An AI-Powered Learning Chatbot for Dynamic Student Engagement.

GIZ5_IND1_G1

Project Members:

1- Hosam Hasan Abdelhady Ibrahim. (Team Leader)

2- Omnia Reda Mohamed Rizk Ghalwash.

3- Mohamed Hussien Mohamed.

4- Doaa Abdelraouf Mahmoud Elganzoury.

Documentation Attribute:

Document Elements	Specification Details
Project Title	Adaptive Learning Assistant: An AI-Powered Learning Chatbot for Dynamic Student Engagement.
Initiative / Program	(DEBI) Industry Graduation Project- Cohort 1.
Core Orchestration Engine	n8n Workflow Automation Engine & LangChain AI Framework (@n8n/n8n-nodes-langchain)
Primary AI & LLM Stack	Google Gemini Chat Model, Text Embeddings & Supabase Vector Store (pgvector).
Database & Storage Layer	Supabase Vector Store (PostgreSQL pgvector HNSW), Dense Vector Embeddings, Google Drive Ingestion.
Communication Channel	Telegram Bot API (Webhook Ingestion, Dynamic Reply Keyboards, Media Stream, Inline Callbacks)
Document Scope	1. Executive Summary & Project Overview | 2. System Architecture | 3. Database Schema | 4. API Documentation | 5. Deployment& Credential Configuration.
Version & Status	Version 1.0 — Final Production Workflow Specification (October 2026).

Production n8n Workflow Topology:

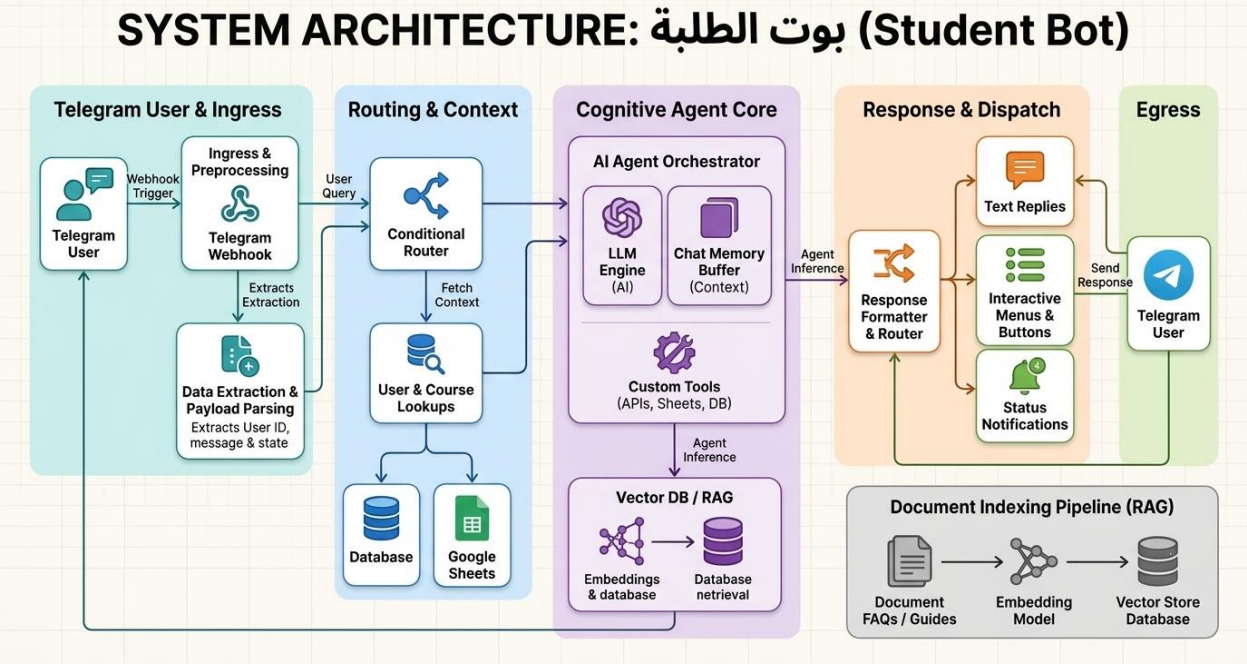


Figure 1.0: Workflow System Architecture — Multi-Branch Student Engagement Engine & RAG Document Ingestion Pipeline.

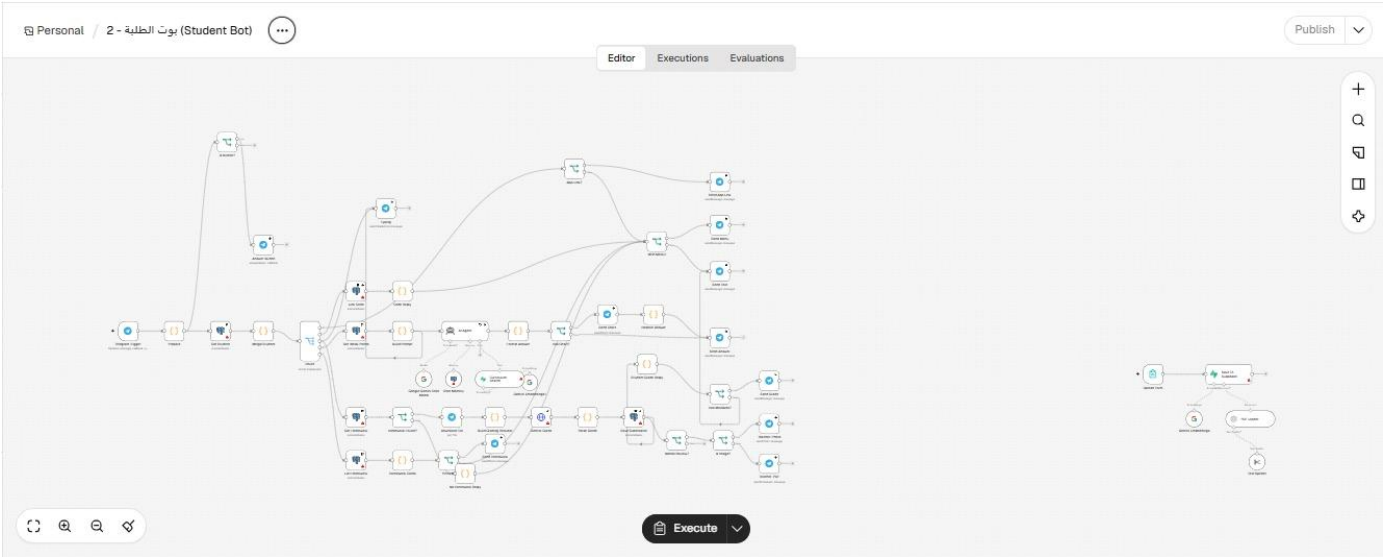


Figure 2.0: The n8n Workflow— Multi-Branch Student Engagement Engine & RAG Document Ingestion Pipeline.

1. Executive Summary & Project Overview.

The Adaptive Learning Assistant (Student Bot) is an enterprise-grade educational workflow engineered in n8n. Designed to eliminate friction in traditional learning management systems, the platform delivers 24/7 personalized pedagogical support, automated assessment scoring, and dynamic curriculum retrieval directly via Telegram. The workflow combines event-driven serverless orchestration with LangChain AI agents, high-dimensional vector search, and dual database persistence.

> **Architectural Vision & Educational Impact**
>
> By uniting n8n's event-driven workflow automation with Google Gemini LLMs, Supabase PostgreSQL/pgvector storage, and the Telegram Bot API, the system delivers a zero-friction learning interface. Students can converse naturally with an AI tutor with curriculum documents, take adaptive quizzes, submit assignments or files, and receive instant diagnostic feedback while all progress metrics are tracked persistently.

1.1 Core Capabilities of the Final n8n Workflow

- **Multi-Path Intent & Event Routing:** Incoming student events from the Telegram Trigger are normalized, authenticated against the PostgreSQL student registry, and routed dynamically across three operational branches via a central Switch Node.
- **Conversational RAG AI Agent:** A LangChain-powered AI Agent node orchestrated within n8n utilizes Google Gemini as its reasoning engine, Postgres Chat Memory for multi-turn conversational continuity, and a Supabase Vector Store tool to retrieve accurate curriculum context.
- **Interactive Assessment & File Processing Pipeline:** A dedicated assessment branch manages student quiz interactions, file/homework downloads via Telegram, dynamic grading logic, database state updates, and conditional outcome notifications.
- **Automated Knowledge Base Ingestion Sub-Workflow:** An independent ingestion pipeline (off-pipeline) monitors Google Drive for new or updated lecture materials, splits documents into semantic chunks, generates vector embeddings via Google Gemini, and indexes them into Supabase.

2. System Architecture.

The system follows a modular, event-driven microservices architecture designed entirely through n8n. The architecture is divided into two primary decoupled subsystems: (A) the Real-Time Student Engagement & Assessment Engine (on- pipeline) and (B) the Asynchronous Knowledge Base RAG Ingestion Pipeline (off- pipeline).

2.1 High-Level Architectural Layers

Architectural Layer	Technologies Used	Primary Responsibilities
1. Presentation & Channel Layer	Telegram Client (Mobile/Desktop/Web) & Telegram Bot API.	Captures student text queries, commands, uploaded files, and callback button presses; renders Markdown replies, quizzes, and feedback.
2. Execution & Routing Layer	n8n Workflow Engine (Webhook Trigger, Switch, IF, Code, Set Nodes).	Ingests webhook events, extracts session metadata, validates user registration state, and routes execution across specialized sub-branches.
3. Cognitive & AI Agent Layer	n8n AI Agent (LangChain Orchestrator), LLM Chat Model, Chat Memory Buffer, Custom Tools.	Performs intent reasoning, tool invocation, semantic query embedding, curriculum synthesis, and personalized pedagogical explanation.
Adaptive Core	Learning Model, Multi-Branch n8n Logic, LLM Grader, Postgres Adaptive State Engine.	Tracks mastery tiers (Beginner, Intermediate, Advanced), evaluates submitted coursework, and decides progression/remediation.
5. Persistence & Memory Layer	PostgreSQL / Supabase (pgvector Extension), LangChain Postgres Chat Memory, Google Drive, Google Sheets	Stores student profile entities, conversation history, vector embeddings, grading archives, and spreadsheet lookups.

2.2 Detailed Node-by-Node Workflow Breakdown

Every node visible in the final n8n workflow diagram performs a deterministic role in processing student requests. The subsections below document the exact sequence of nodes across the entry pipeline, the three routing branches, and the secondary vector ingestion workflow.

A. Entry & Pre-Processing Pipeline.

Seq	n8n Node Name / Type	Component Role & Data Transformation Logic
1	Telegram Trigger (n8n-nodes-base.telegramTrigger)	Listens for real-time HTTPS webhook POST payloads from the Telegram Bot API (message, edited_message, callback_query, document, voice).
2	Set / Extract Fields (n8n-nodes-base.set / code)	Normalizes raw Telegram payload variations into standardized workflow variables: telegram_id, chat_id, username, full_name, message_text, message_type, and file_id.
3	Postgres: Get Student Profile (n8n-nodes-base.postgres)	Executes a SELECT / UPSERT query against the students table using telegram_id to retrieve the student's current learning level, active session state, and registration status.
4	Prepare Routing Context (n8n-nodes-base.code)	Evaluates student profile state, command prefixes (/start, /quiz, /help), attachment flags, and active session locks to compute a deterministic route_key.
5	Switch: Master Router (n8n-nodes-base.switch)	Splits workflow execution into three distinct operational branches: Branch 1 (Top: Onboarding/Profile), Branch 2 (Middle: RAG AI Agent), and Branch 3 (Bottom: Quiz/File Evaluation).

B. Branch 1: Student Onboarding, Command & State Management

When the Switch node detects a first-time user (/start), a profile update command, or a system state reset, execution is routed to the top branch:

- **Postgres State Node (Set/Update Student):** Inserts a new student record or updates the student's current module/topic preferences in PostgreSQL.
- **Format Welcome / Status Message (Code/Set Node):** Constructs a personalized onboarding greeting or dashboard summary displaying the student's current progress and available commands.
- **Telegram Send Message (Top Convergence Node):** Delivers the formatted onboarding or status response directly to the student's Telegram chat.

C. Branch 2 (Middle Path): Adaptive AI Tutor & RAG Agent Pipeline

When a student submits a natural-language academic question or conceptual inquiry, the Switch node routes the payload to the middle branch, which houses the core LangChain AI Agent cluster:

Node / Sub-Node	n8n Node Type	Technical Function in RAG Pipeline
Pre-Agent Context Log	Postgres + Code Nodes	Logs the incoming student question and injects student profile metadata (learning style, difficulty tier, active course) into the agent prompt.
AI Agent (Core)	@n8n/n8n-nodes-langchain.agent	Orchestrates reasoning using a system prompt tailored for Socratic tutoring and dynamic student engagement.
Google Gemini Chat Model	@n8n/n8n-nodes-langchain.lmChatGoogleGemini	Connected as the Model sub-node (`ai_languageModel`). Generates context-aware educational responses using Gemini (e.g., gemini-1.5-pro / gemini-2.0-flash).
Postgres Chat Memory	@n8n/n8n-nodes-langchain.memoryPostgresChat	Connected as the Memory sub-node (`ai_memory`). Persists and retrieves recent conversation turns keyed by chat_id for multi-turn continuity.
Supabase Vector Store Tool	@n8n/n8n-nodes-langchain.vectorStoreSupabase	Connected as a Tool (`ai_tool`). Executes cosine similarity search against the course knowledge base when the agent needs verified curriculum facts.
Embeddings Google Gemini	@n8n/n8n-nodes-langchain.embeddingsGoogleGemini	Connected beneath the Vector Store tool (`ai_embedding`). Converts the agent's search query into a 768-dimensional vector representation.
Post-Agent Response Handler	Set -> IF -> Telegram -> Code	Formats the AI Agent output, checks message length/validity via an IF node, sends intermediate or rich responses via Telegram, and converges to the master Telegram sender.

D. Branch 3: Interactive Quiz, File Submission & Evaluation Pipeline

The bottom branch handles structured learning activities, specifically interactive quizzes, assignment file uploads, and automated grading workflows:

Stage	Workflow Node Sequence	Operational Logic
1. Session & Input Check	Postgres Node -> IF Node	Retrieves active quiz/assignment session details from PostgreSQL and checks whether the student input is a valid submission or requires a fallback prompt.
2. Media / File Retrieval	Telegram Node (Get File) + Fallback Code Node	If the student uploaded a document/image or triggered a specific quiz payload, the Telegram node fetches the file metadata/binary; otherwise, the lower Code node formats a retry/help prompt.
3. Evaluation & Grading	Code Node -> OpenAI / LLM Grader -> Code Parser	Prepares the grading rubric, invokes a dedicated LLM evaluation node to assess the student's answer or extracted submission, and parses the structured JSON score and feedback.
4. Score Persistence	Postgres Update Node	Writes the student's score, mastery delta, and detailed diagnostic feedback into the quiz_attempts and student_progress tables.
5. Mastery Routing	Code Node + IF Split -> Dual Telegram Nodes	Evaluates whether the score meets the passing mastery threshold (e.g., >= 70%). Routes to either the Top Telegram Node (Pass/Next Level Unlocked) or Bottom Telegram Node (Remediation/Retry Guidance).

E. Secondary Workflow (offline- Pipline): Knowledge Base Vector Ingestion Pipeline

Positioned on the right side of the n8n canvas is the autonomous RAG ingestion pipeline responsible for keeping the chatbot's curriculum knowledge base synchronized with official DEBI course materials:

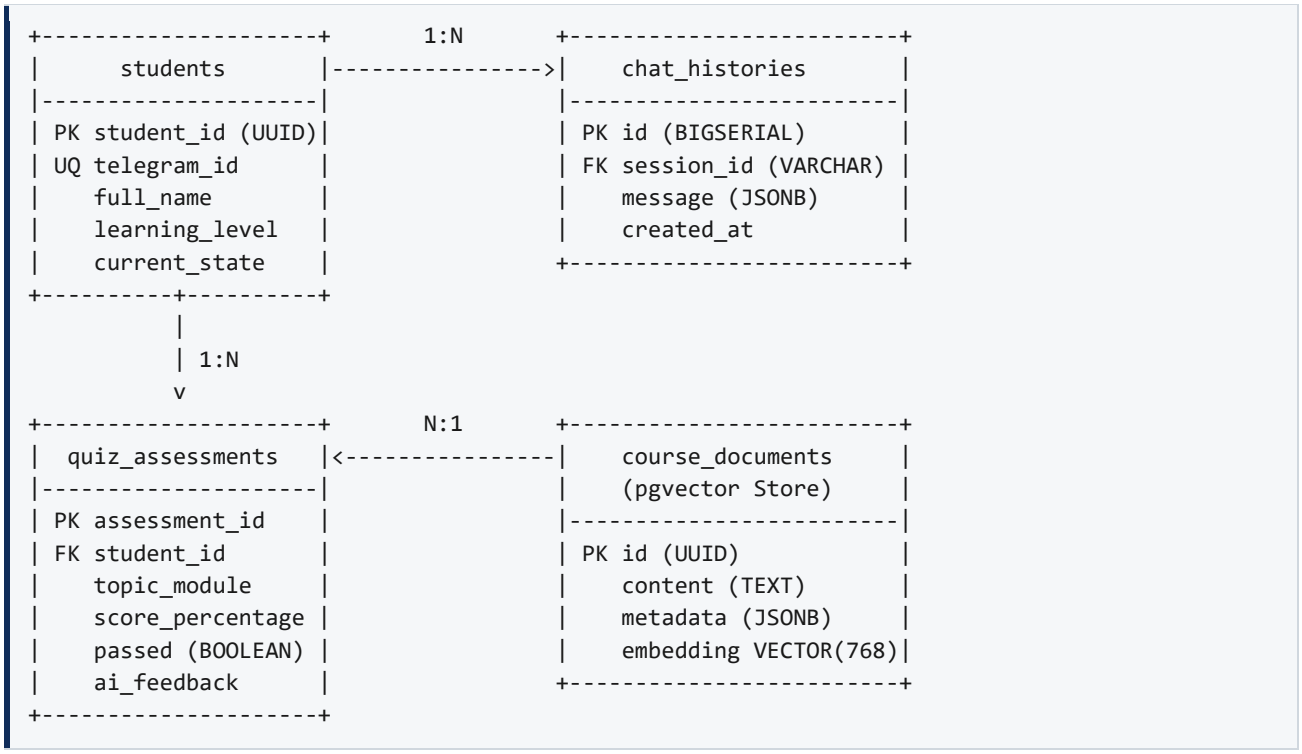
1. **Google Drive Trigger / File Loader Node:** Detects newly uploaded or modified curriculum documents (PDFs, lecture notes, slides) in the designated Google Drive folder and downloads the binary stream.
2. **Supabase Vector Store (Insert Mode):** Coordinates the ingestion of document chunks and metadata into the PostgreSQL pgvector table (`course_documents`).

3. Embeddings Google Gemini Sub-Node: Generates dense 768-dimensional semantic vectors for every document chunk using Google's `text-embedding-004` model.
4. Default Data Loader Sub-Node: Extracts raw text and structural metadata (file name, module ID, page number) from binary PDF/Docx files.
5. Recursive Character Text Splitter Sub-Node: Splits extracted text into overlapping semantic chunks (e.g., chunk size = 1,000 characters, chunk overlap = 200 characters) to preserve context across boundaries.

3. Database Schema Design.

The persistence layer is built on Supabase (managed PostgreSQL 15+ with the pgvector extension enabled). This unified database architecture allows the n8n workflow to execute relational CRUD operations (via standard Postgres nodes), conversational memory persistence (via the LangChain Postgres Chat Memory node), and high-speed semantic vector similarity searches (via the Supabase Vector Store node) within a single transactional environment.

3.1 Entity-Relationship Architecture Overview



3.2 Relational & Vector Table Specifications

Table 1: students (Core Student Registry & Adaptive State)

Accessed by the entry Postgres and updated across Branches 1 and 3 to track student identity, adaptive difficulty level, and current workflow state.

Column Name	Data Type	Constraints	Description
student_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique internal identifier for each registered student.
telegram_id	BIGINT	UNIQUE, NOT NULL, INDEXED	Telegram user ID extracted from `message.from.id`.
chat_id	BIGINT	NOT NULL	Telegram chat channel ID used for outbound bot messaging.
username	VARCHAR(100)	NULLABLE	Telegram handle (`@username`) for administrative reference.
full_name	VARCHAR(255)	NOT NULL	Student's concatenated first and last name.
track_name	VARCHAR(100)	DEFAULT 'DEBI AI & Data Science'	Assigned academic track within the DEBI program.
learning_level	VARCHAR(30)	DEFAULT 'Beginner' CHECK (IN ('Beginner','Intermediate','Advanced'))	Dynamic mastery tier adjusted automatically based on quiz performance.
current_state	VARCHAR(50)	DEFAULT 'IDLE'	Workflow state machine flag (`IDLE`, `IN_QUIZ`, `AWAITING_FILE`, `ONBOARDING`).
total_points	INTEGER	DEFAULT 0	Cumulative gamification and assessment score across modules.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp of initial student registration.
last_active_at	TIMESTAMPTZ	DEFAULT NOW()	Updated on every incoming webhook interaction.

Table 2: n8n_chat_histories (LangChain Postgres Chat Memory)

Automatically managed by the `@n8n/n8n-nodes-langchain.memoryPostgresChat` node attached to the AI Agent to maintain multi-turn conversational context.

Column Name	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Sequential auto-incrementing message log ID.
session_id	VARCHAR(128)	NOT NULL, INDEXED	Unique conversation key mapped to the student's `telegram_id`.
message	JSONB	NOT NULL	LangChain serialized message object (`type`: `human` or `ai`, `content`, `tool_calls`).
created_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp when the memory turn was recorded.

Table 3: course_documents (Supabase pgvector RAG Knowledge Store)

Populated by the right-hand Google Drive ingestion pipeline and queried via semantic similarity by the AI Agent's Supabase Vector Store tool.

Column Name	Data Type	Constraints	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique identifier for each embedded text chunk.
content	TEXT	NOT NULL	Raw curriculum text chunk produced by the Recursive Character Text Splitter.
metadata	JSONB	NOT NULL, DEFAULT '{}'	Contains `file_id`, `file_name`, `module_number`, `page_number`, and `difficulty`.
embedding	VECTOR(768)	NOT NULL, HNSW INDEX	768-dimensional dense embedding generated by Google Gemini `text-embedding-004`.
indexed_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp when the chunk was ingested from Google Drive.

Table 4: quiz_assessments & submissions (Branch 3 Evaluation Store)

Stores structured quiz interactions, uploaded file evaluations, AI grading rationales, and mastery progression logs generated in the bottom branch.

Column Name	Data Type	Constraints	Description
assessment_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Unique assessment attempt identifier.
student_id	UUID	FOREIGN KEY -> students(student_id)	Links the assessment record to the student profile.
assessment_type	VARCHAR(40)	NOT NULL	Indicates `INTERACTIVE_QUIZ`, `SHORT_ANSWER`, or `FILE_SUBMISSION`.
module_topic	VARCHAR(150)	NOT NULL	Curriculum topic or lecture module being evaluated.
student_input	TEXT	NULLABLE	Student's submitted answer text or extracted file content.
telegram_file_id	VARCHAR(255)	NULLABLE	Telegram `file_id` reference if the submission included a document.
score_percentage	NUMERIC(5,2)	NOT NULL	Calculated percentage score (0.00 to 100.00) from the AI evaluator.
passed	BOOLEAN	NOT NULL	True if `score_percentage >= 70.00` (triggers Pass Telegram branch).
ai_feedback	TEXT	NOT NULL	Personalized strengths, weaknesses, and remediation hints generated by the LLM.
evaluated_at	TIMESTAMPTZ	DEFAULT NOW()	Timestamp of assessment completion.

3.3 Production PostgreSQL / Supabase DDL Script

```
-- Enable pgvector extension for semantic search
CREATE EXTENSION IF NOT EXISTS vector;

-- 1. Students Profile & State Table
CREATE TABLE IF NOT EXISTS public.students (
    student_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    telegram_id BIGINT UNIQUE NOT NULL,
    chat_id BIGINT NOT NULL,
    username VARCHAR(100),
    full_name VARCHAR(255) NOT NULL,
    track_name VARCHAR(100) DEFAULT 'DEBI AI & Data Science',
    learning_level VARCHAR(30) DEFAULT 'Beginner'
        CHECK (learning_level IN ('Beginner', 'Intermediate', 'Advanced')),
    current_state VARCHAR(50) DEFAULT 'IDLE',
    total_points INTEGER DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    last_active_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_students_telegram_id ON public.students(telegram_id);

-- 2. LangChain Chat Memory Table
CREATE TABLE IF NOT EXISTS public.n8n_chat_histories (
    id BIGSERIAL PRIMARY KEY,
    session_id VARCHAR(128) NOT NULL,
    message JSONB NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_chat_histories_session ON public.n8n_chat_histories(session_id);

-- 3. Course Knowledge Base Vector Table (768-dim Gemini Embeddings)
CREATE TABLE IF NOT EXISTS public.course_documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    content TEXT NOT NULL,
    metadata JSONB NOT NULL DEFAULT '{}'::jsonb,
    embedding VECTOR(768) NOT NULL,
    indexed_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_course_docs_embedding ON public.course_documents
USING hnsw (embedding vector_cosine_ops);

-- 4. Supabase RPC Function for Semantic Similarity Search
CREATE OR REPLACE FUNCTION match_course_documents (
    query_embedding VECTOR(768),
    match_threshold FLOAT DEFAULT 0.72,
    match_count INT DEFAULT 5,
    filter JSONB DEFAULT '{}'::jsonb
) RETURNS TABLE (
    id UUID,
    content TEXT,
    metadata JSONB,
    similarity FLOAT
) LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        cd.id,
        cd.content,
```

```
        cd.metadata,
        1 - (cd.embedding <=> query_embedding) AS similarity
    FROM public.course_documents cd
    WHERE 1 - (cd.embedding <=> query_embedding) > match_threshold
      AND cd.metadata @> filter
    ORDER BY cd.embedding <=> query_embedding
    LIMIT match_count;
END;
$$;
```

4. API & Integration Documentation

The Adaptive Learning Assistant operates as an event-driven hub integrating four primary external and internal APIs: (1) Telegram Bot Webhook & Messaging APIs, (2) Supabase REST & Vector RPC APIs, (3) Google Gemini Generative AI & Embeddings APIs, and (4) Google Drive Document Synchronization APIs.

4.1 Inbound Webhook API: Telegram Trigger Endpoint

The n8n Telegram Trigger node registers an HTTPS webhook endpoint with the Telegram Bot API. Whenever a student sends a message, presses an inline button, or uploads an assignment file, Telegram dispatches a JSON POST payload to n8n.

Property	Specification
Endpoint URL	POST https://<n8n-domain>/webhook/<telegram-webhook-uuid>/webhook
Authentication	X-Telegram-Bot-Api-Secret-Token header validation
Content-Type	application/json
Supported Update Types	message (text, document, photo, voice), callback_query (inline quiz buttons)

Sample Inbound Webhook Payload (Student Query / File Upload)

```
{
  "update_id": 849302114,
  "message": {
    "message_id": 1420,
    "from": {
      "id": 519283746,
      "is_bot": false,
      "first_name": "Omar",
```

```
      "last_name": "Hassan",
      "username": "omar_debi",
      "language_code": "en"
    },
    "chat": {
      "id": 519283746,
      "first_name": "Omar",
      "last_name": "Hassan",
      "type": "private"
    },
    "date": 1760798400,
    "text": "Can you explain how backpropagation updates weights in a neural network?"
  }
}
```

4.2 Outbound Telegram Bot API Operations

Across the three workflow branches, five Telegram action nodes interact with `https://api.telegram.org/bot<TOKEN>/...` to deliver responses, download student files, and dispatch assessment results.

Workflow Branch / Node	Telegram API Method	Endpoint & Payload Parameters
Master & Mid-Branch Reply Nodes	sendMessage	POST /sendMessage Body: { "chat_id": 519283746, "text": "...", "parse_mode": "Markdown", "reply_markup": {...} }
Bottom Branch: File Downloader	getFile & File Stream	GET /getFile?file_id=<file_id> Downloads binary from https://api.telegram.org/file/bot<TOKEN>/<file_path>
Bottom Branch: Pass Notification	sendMessage (Mastery Pass)	POST /sendMessage Sends congratulatory mastery badge, updated points, and next module unlock button.
Bottom Branch: Remediation Alert	sendMessage (Retry/Review)	POST /sendMessage Sends targeted explanation of missed concepts and a 'Retry Quiz' inline keyboard.

4.3 Internal AI Agent & Vector Search API Contract (Supabase RPC)

When the LangChain AI Agent invokes the Supabase Vector Store tool, n8n first calls the Google Gemini Embeddings API to vectorize the query, then invokes the Supabase PostgREST RPC endpoint to retrieve the top-k relevant curriculum chunks.

```
// 1. Request to Supabase Vector RPC: POST https://<project-
ref>.supabase.co/rest/v1/rpc/match_course_documents
// Headers: apikey: <SUPABASE_SERVICE_ROLE_KEY>, Authorization: Bearer
<SUPABASE_SERVICE_ROLE_KEY>
{
  "query_embedding": [0.0142, -0.0318, 0.0491, "...", 0.0087],
  "match_threshold": 0.72,
  "match_count": 4,
  "filter": {
    "track": "DEBI AI & Data Science"
  }
}

// 2. Response Payload Returned to AI Agent Tool:
[
  {
    "id": "c9a4f812-3b7d-4e91-8a15-92d8f01a4c33",
    "content": "Module 4 - Neural Networks: Backpropagation computes the gradient of the loss
function with respect to each weight by applying the chain rule layer by layer from output to
input...",
    "metadata": {
      "file_name": "DEBI_Deep_Learning_Lecture_04.pdf",
      "module_number": 4,
      "page_number": 18,
      "difficulty": "Intermediate"
    },
    "similarity": 0.894
  }
]
```

4.4 Automated Grading & Evaluation JSON Contract (Bottom Branch)

In Branch 3, the evaluation LLM node is instructed to return a strict JSON schema so the downstream Code and IF nodes can deterministically update PostgreSQL and route between the Pass and Remediation Telegram nodes:

```
{
  "student_id": "7f8b2c10-9e4a-4d12-b831-001928374655",
  "module_topic": "Neural Networks & Backpropagation",
  "score_percentage": 85.0,
  "passed": true,
  "recommended_level": "Intermediate",
  "points_awarded": 25,
  "feedback_summary": "Excellent explanation of the chain rule and gradient descent step! To
reach 100%, remember to mention how vanishing gradients affect deep sigmoid networks.",
  "next_action": "UNLOCK_NEXT_MODULE"
}
```

4.5 Error Handling, Rate Limiting & Security Specifications

HTTP / System Code	Trigger Condition in Workflow	Automated n8n Recovery & Fallback Mechanism
400 Bad Request	Malformed Telegram Markdown or invalid callback query payload.	Code node sanitizes special Markdown characters (`_`, `*`, `[`) and retries delivery in plain text mode.
401 / 403 Unauthorized	Expired Google OAuth2 token or invalid Supabase/Telegram API key.	n8n credential manager refreshes OAuth2 tokens automatically; alerts administrator channel on hard auth failure.
422 Unprocessable Input	Student uploads an unsupported file format or empty message in Branch 3.	IF node triggers the lower fallback Code node (`Format Fallback Reply`) and prompts the student with supported formats.
429 Too Many Requests	Google Gemini API quota burst or Telegram flood limit exceeded.	n8n node-level 'Retry On Fail' enabled (Max Tries: 3, Wait Between Tries: 2000ms exponential backoff).
500 / DB Timeout	Transient PostgreSQL connection drop or LLM parsing failure.	Graceful fallback message sent via Telegram informing the student to retry in a few seconds while preserving chat state.

5. Deployment & Credential Configuration Matrix

To deploy or replicate the Adaptive Learning Assistant n8n workflow across staging and production environments, the following credentials and environment variables must be configured in the n8n instance:

Credential Name in n8n	Target Service	Required Scopes / Configuration Parameters
Telegram Bot API	Telegram Trigger & Action Nodes	Bot Token issued by @BotFather. Webhook registered with HTTPS SSL certificate.
PostgreSQL Account	Postgres Nodes & Chat Memory	Host: db.<project>.supabase.co, Port: 5432, Database: postgres, SSL: Require.
Supabase API	Supabase Vector Store Nodes	Project URL and service_role Secret Key for vector CRUD and HNSW searches.
AI Studio / Model Key	Chat Model & Embeddings Sub-Nodes	API Key with active quota for conversational reasoning and dense vector embeddings.
Google Drive OAuth2	Drive Document Ingestion Trigger	Client ID & Secret with drive. Read only scope restricted to the course syllabus folder.